UTM

UNIVERSITI TEKNOLOGI MALAYSIA

BACHELOR OF COMPUTER SCIENCE (DATA ENGINEERING) WITH HONS.

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3213 - BUSINESS INTELLIGENCE

PROJECT (PHASE 2)

GROUP : SKINAQUA

PREPARED BY :

NAME	MATRIC NO
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
AIN NURNABILA BINTI MOHD AZHAR	A23CS0207
SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258

PREPARED FOR:

DR. NOORFA HASZLINA BINTI MUSTAFFA

TABLE OF CONTENT

1.0 Introduction

The phase 2 of the Business Intelligence Solution for Retail Pricing and Demand Analysis project is moving from planning into implementation by designing and constructing the complete data preparation workflow in Alteryx Designer. The aim of this phase is to take all three separate retail datasets and turn them into a single, clean, analysis ready dataset that can be passed directly to Power BI/Tableau for the visualization work in phase 3.

The workflow integrates three structured datasets sourced from Kaggle, which are the Retail Pricing & Demand Signals datasets serving as the base table, holding product level pricing, discount, inventory, and demand observations. Next, the Retail Product Catalogue dataset is joined to enrich each observation with product name, brand, category, and rating metadata. Lastly, the Product Sales dataset which covers transaction-level order records.

The workflow in this phase was built to cover the full data preparation pipeline required for this phase which involves reading data from multiple sources, cleaning each dataset to handle data quality issues, and finally integrating the prepared branch into one consolidated output. Each dataset is prepared on its own branch before merging, so the issue in one source can be corrected without affecting the others.

2.0 Business Intelligence (BI) architecture

As shown in Figure 2.1 is the BI architecture that is used in this project. The architecture has five layers:

1. Data source layer

   The three raw dataset from Kaggle acted as the data source in the architecture.

2. Data preparation

   There are four inner stages inside this layer, which are input data (inserting multiple data inputs) → clean (remove nulls, duplicates, invalid records, type fixes) → transform (the selected derived fields) → integrate (join between datasets) .

3. Data storage

   Alteryx exports one consolidated, analysis ready CSV file for the next layer.

4. Visualization

   Using the saved CSV files, dashboards and storyboards in generated using PowerBI/Tableau.

5. Decision layer

   This layer turns a data flow into a proper BI architecture, since BI is defined by decision support

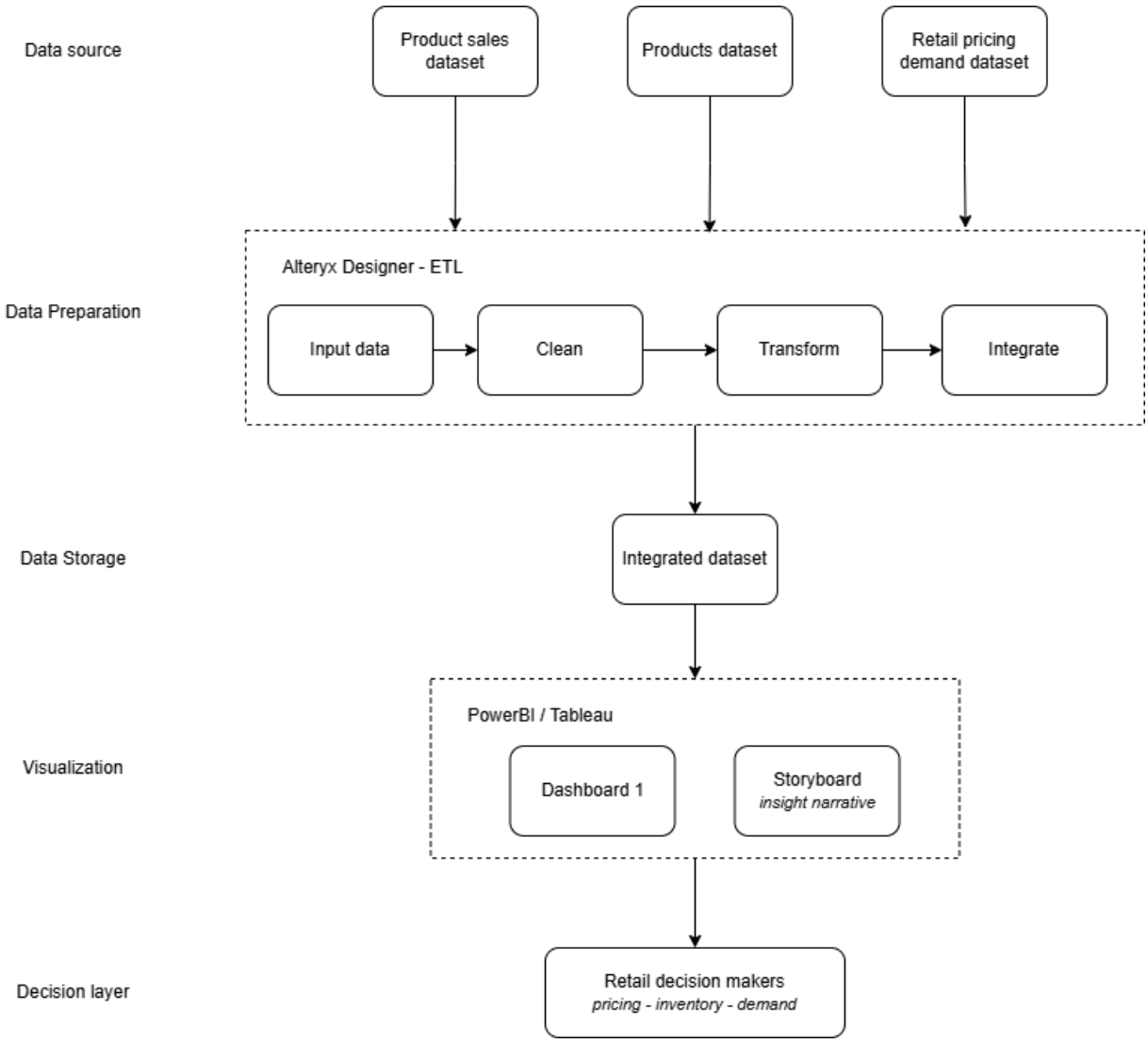

Figure 2.1 BI Architecture

3.0 Alteryx tools used

Table 3.1 shows lists of alteryx tools that are used and its role in this workflow.

Table 3.1 Alteryx tools used

Tools	**Role in the workflow**
Input data	Loads the three source CSV files
Select	Selects fields and corrects data types
Dynamic Rename	Trims whitespace from field names
DateTime	Converts Order_Date text to a date field
Data Cleansing	Trims whitespace and standardises text case
Unique	Removes duplicate records per branch
Filter	Removes invalid (non-positive) records
Formula	Standardises category labels
Summarize	Aggregates sales to category-month level
Join	Integrates the datasets on product_id and category
Union	Consolidates the integration output
Browse	Inspects and verifies the final dataset
Output Data	Writes the integrated dataset to CSV for Tableau

4.0 Data Cleaning

A vital step in any Business Intelligence pipeline is one that is known as Data Cleaning. All three datasets were first checked for data quality problems, such as nulls, duplicate data, invalid values, and data type mismatches, before they could be combined and then analysed. If data was left dirty, it would cause errors to pass down the stream to the integration and transformation phases resulting in wrong visualisations and erroneous business insights. The cleaning strategy for each individual dataset, done in Alteryx Designer, is documented here.

A cleaning pipeline is three tools, applied one after the other, to each dataset: a Data Cleansing tool to deal with blank values and spaces, a Unique tool to remove duplicate records, and a Filter tool to exclude any invalid rows. The tools were hooked up following Member 1's (Yasmin's) Select tools which had previously confirmed and given the correct data types to each field.

4.1 Data Quality Findings Summary

These pre-cleaning audits of all three datasets yielded the following results:

Table 4.1.1 *Pre-cleaning audit on all three datasets*

Dataset	File	Rows	Null Values Found	Duplicated Found	Invalid Records
DS1 (Retail Pricing & Demand)	retail_pricing_demand_100k.csv	172,800	100,155 in promotion_type (58.0%)	0	9 rows (revenue = 0, units_sold = 0)
DS2 (Product Catalog)	products.csv	20	None	0	None
DS3 (Product Sales 2023-2024)	product_sales_dataset_final.csv	200,000	None	0	None

4.2 DS1 – Retail Pricing & Demand Signals

Data Cleansing Tool

The most impactful data quality problem found was for the promotion_type column in the Retail Pricing & Demand dataset. Out of 172,800 rows, 100,155 records (58.0%) had null values in this field. This is normal—if there is no current promotion for a product, there will be no promotion type. To ensure that this did not discard 58% of the data (and remove valid pricing observations), the Data Cleansing tool was set up to replace any nulls in the promotion_type column with the value 'None''.

This ensures that the entire data set and also the lack of a promotion are made apparent and can be utilized in additional downstream analyses. Leading and trailing whitespace has been trimmed from all string fields too, to allow for cleaner string comparisons when joining.

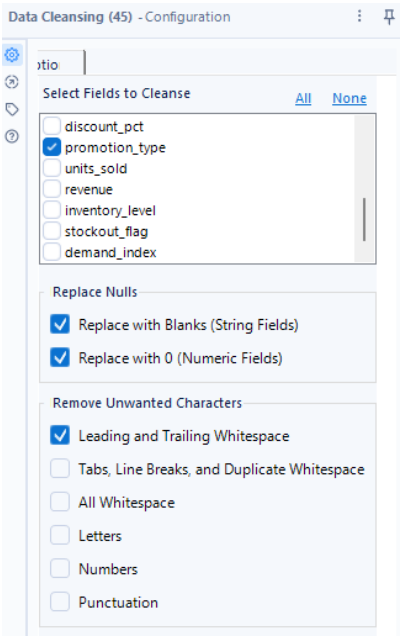

Figure 4.2.1 *Data Cleansing configuration tool, only promotion_type field are selected*

Figure 4.2.2 *Null values are replaced with 'None'*

Unique Tool

The Unique tool has been set up with a key of date + product_id + region. The three-field key guarantees that only one price observation is maintained for each product,

each date and each region, as the data naturally consists of these three fields. The audit found no duplicate rows for this key, showing that a total of 172,800 rows are unique combinations of product, date and region. Records were silently discarded because the Duplicate output port of the Unique tool was not connected.

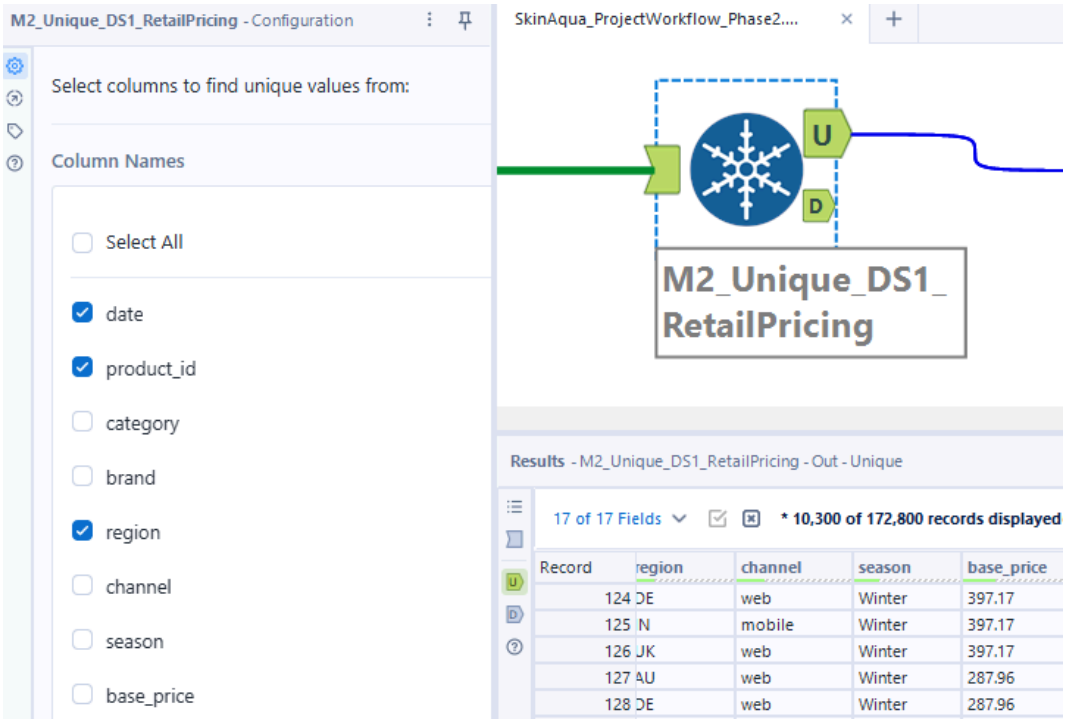

Figure 4.2.3 *Unique configuration tool; date, product_id, and region fields are checked*

Filter Tool

Records for events with zero units sold and zero revenue (zero sales events) have been filtered out: [units_sold] >= 0 AND [revenue] > 0. During the audit, 9 were found in 6 product categories (Shoes, Apparel, Accessories, Beauty, Groceries, Home). These rows are not used for analyses of sales trends or for modelling the demand for the product and so were not included in the clean output. 172,791 rows were passed to the integration stage, after filtering.

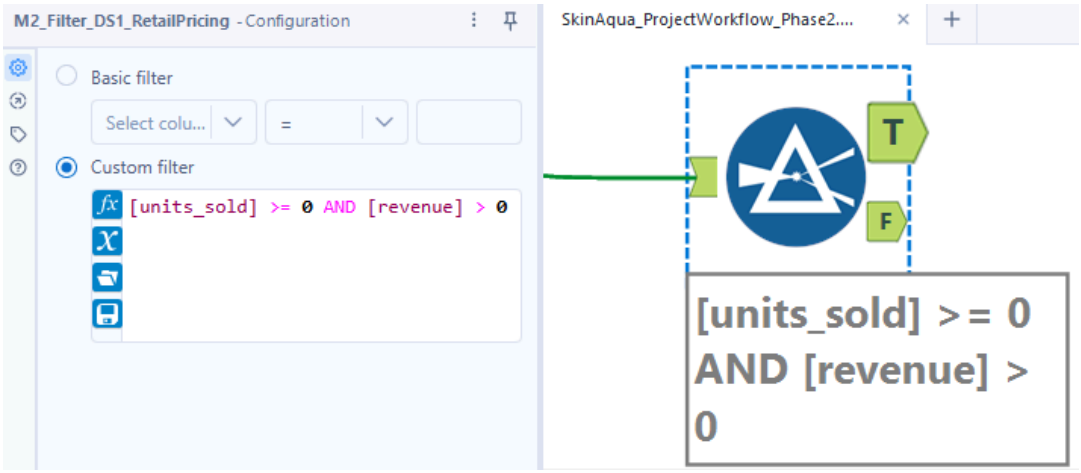

Figure 4.2.4 *Filter configuration tool, filter expression [units sold] >= 0 AND [revenue] = 0*

4.3 DS2 – Retail Product Catalog

The Product Catalog data set (products.csv) was completely clean and had no null values, no duplicate product numbers, and no invalid data in any of its 20 rows and 11 columns during pre-cleaning. Even so, the Data Cleansing and Unique tools were also used as a protective barrier to keep the data set clean even if any changes are made to the source file at the top of the data flow.

The Data Cleansing tool was set to be applied to all fields, trimming any leading or trailing whitespace, and replacing any null or empty strings with “Unknown”. Utilizing the Unique tool keyed the product_id and each product would only show up once in the catalog before it was joined to the pricing dataset. All 20 records went through both tools and passed without any modifications, which shows that the data from which both records were created is intact.

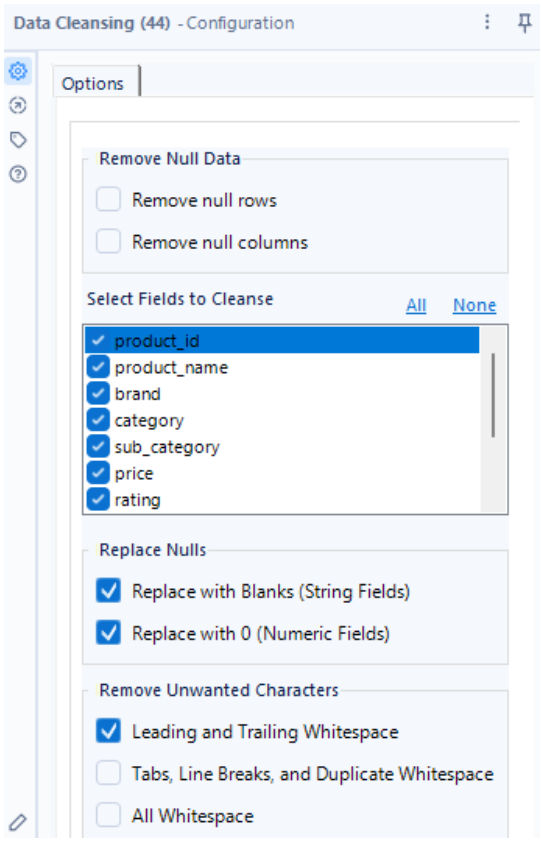

Figure 4.2.5 *Data Cleansing configuration tool, all fields are selected*

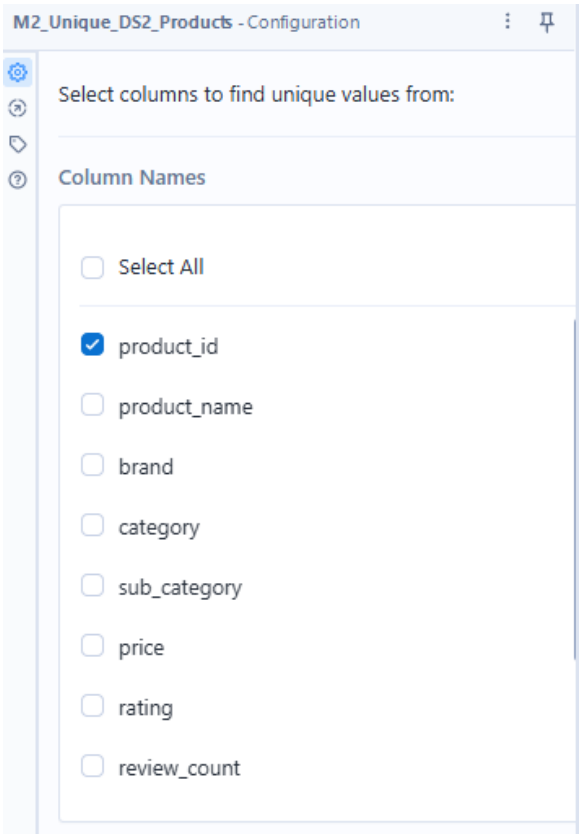

Figure 4.2.6 *Unique configuration tool, only product_id are selected*

4.4 DS3 – Product Sales Dataset (2023 - 2024)

The Product Sales dataset (product_sales_dataset_final.csv) was also clean with no nulls, no duplicate Order IDs, and no anomalies in revenue or quantity, in all 200,000 rows. But one problem was found in Member 1's DateTime tool (Tool 10): When the Order_Date values are in the format MM-dd-yy (for example 08-23-23 for August 23, 2023), it is not displayed in the correct format (2023-08-23). This is addressed in the current workflow file prior to any cleaning tools.

The Data Cleansing tool was set up to run on all fields, stripping out any whitespace and replacing any null/blank strings with “Unknown”. The primary transaction identifier, Order_ID, was used for the Unique tool. The Filter tool used the criteria [Revenue] > 0 and [Quantity] > 0 to remove any rows that have a zero revenue entry or quantity entry. In the pre-cleaning audit,

there are no records failing this filter, and thus all of the 200,000 rows were kept in the clean output.

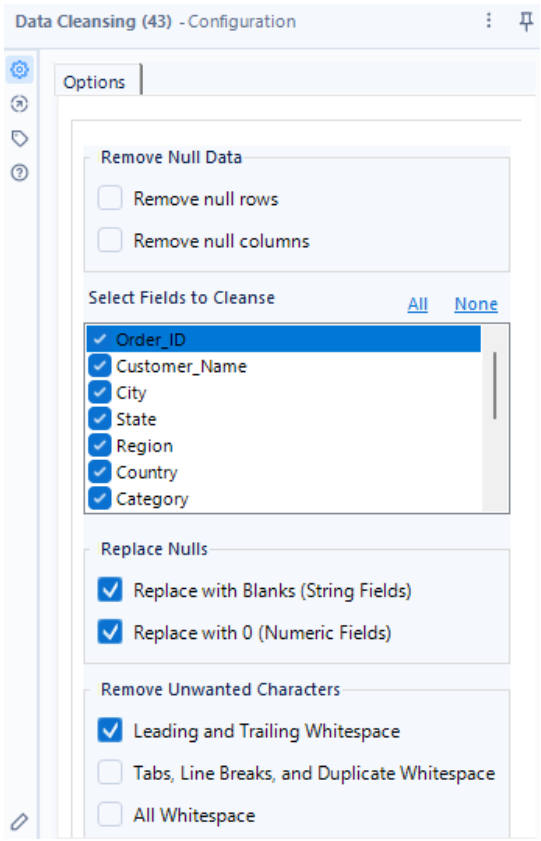

Figure 4.2.7 *Data Cleansing configuration tool, all fields are selected*

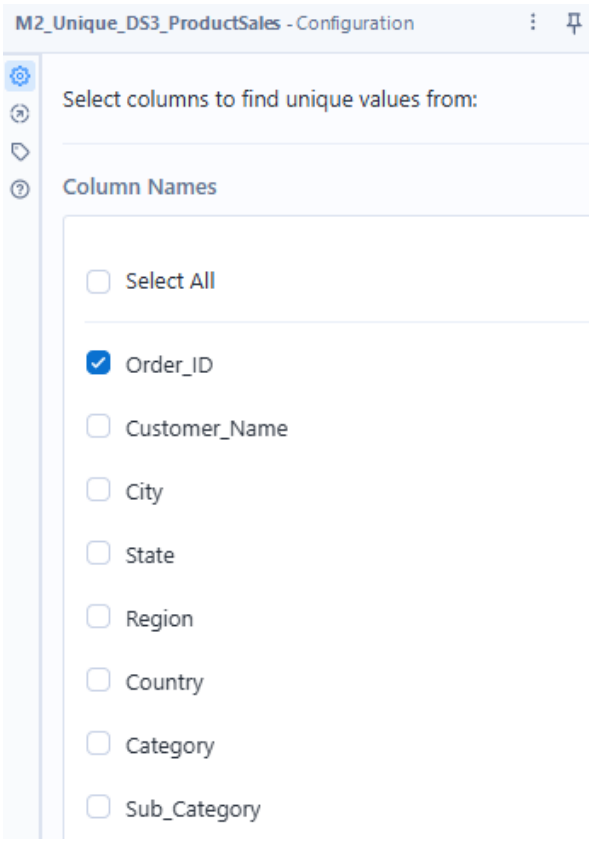

Figure 4.2.8 *Unique configuration tool, only Order_ID are selected*

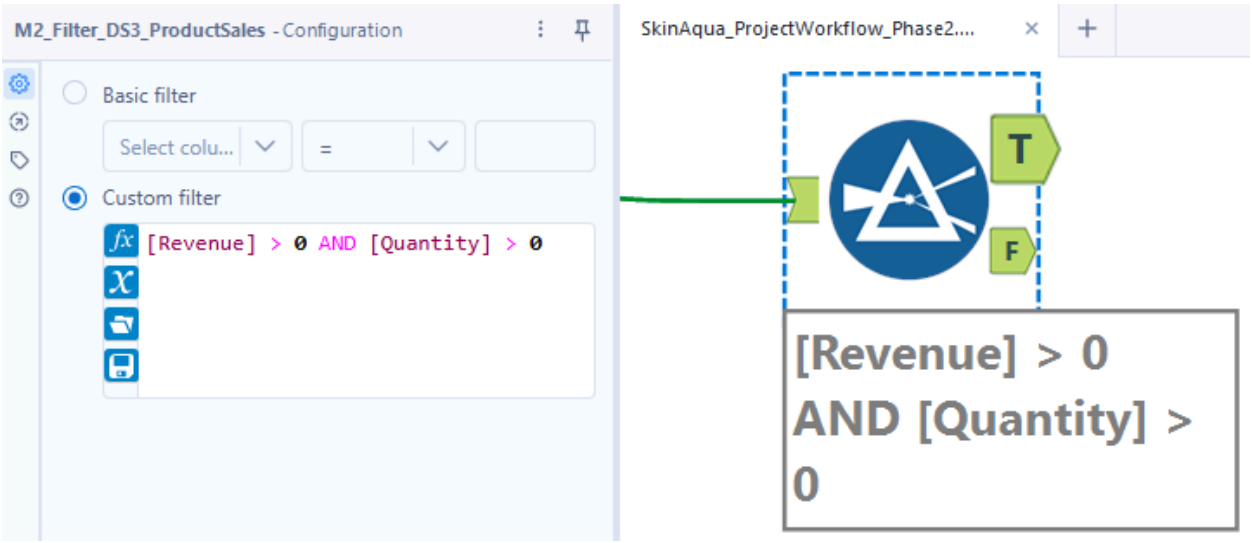

Figure 4.2.4 *Filter configuration tool, filter expression [Revenue] > 0 AND [Quantity] > 0*

4.5 Why Data Cleaning Matter for This Project

Data cleanliness isn't just a formality; it's a key requirement for reliable business intelligence. The main goal of this project is to determine the price elasticity, seasonal trends and slow moving stock in product groups. If the promotion_type attributes were left as nulls in the DS1, then any pivot table or visualisation that segmented the data by promotion type would either show 58% of the data at a null level or would represent a vague null level, meaning that each analysis of promotion type in the pivot table would be skewed. Likewise, although there are only nine zero-revenue records in DS1, these are structurally distinct records (no sales activity) that should not be combined with sales activity records in determining demand indices or revenue trends.

This stage was the point at which cleaning was done, after data input and type confirmation, but before integration, in accordance with the principle of cleaning in the earliest possible point in the pipeline. This way, any joins, transformations and aggregations in later stages are done on analytically meaningful, consistent and complete data.

5.0 Data Integration

5.1 Overview of Integration Approach

The data integration phase combines all three cleaned datasets into a single unified table that can be passed to Member 4 for transformation and output. The integration is performed in two sequential join operations rather than attempting to merge all three datasets simultaneously. This staged approach ensures that join logic remains clear, errors can be isolated per join, and data lineage is easy to trace.

The first join operates at the product level, merging DS1 (Retail Pricing & Demand Signals) with DS2 (Retail Product Catalog) on the shared product_id field. The second join operates at the category level, enriching the merged DS1+DS2 table with aggregated sales figures from DS3 (Product Sales 2023–2024). DS3 is first summarised by category and order month before joining, as it contains order-level records that would otherwise cause row multiplication during the join.

5.2 Join 1 (DS1 + DS2 at Product Level)

Join key: product_id (DS1) = product_id (DS2)

Join type: Left Join

A Left Join was selected to ensure that all records from DS1 are preserved in the output, even where no matching product exists in the DS2 catalog. DS1 is the primary analysis dataset containing pricing signals and demand metrics, so none of its rows should be discarded due to a missing catalog entry as shown in Figure 5.1.

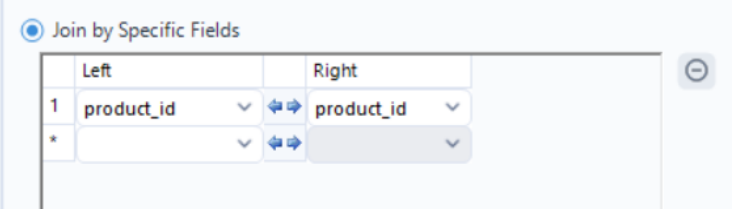

Figure 5.1 *Join 1 configuration panel with join condition: DS1.product_id = DS2.product_id*

5.3 Select Tool (To remove duplicate columns)

A Select tool is placed immediately after Join 1 to remove any columns that were duplicated across DS1 and DS2, such as a second instance of the brand or category field originating from DS2. This keeps the schema clean before the category standardisation step. Since Join 1 returns no matched rows in this workflow, the Select tool primarily serves as a structural safeguard to prevent column name conflicts in downstream tools.

5.4 Category Name Standardisation (Formula Tool)

Before performing Join 2, the category field in the DS1+DS2 merged table must be standardised to match the category naming convention used in DS3. Each of the three datasets was authored independently on Kaggle and uses a different set of category labels, as shown in the Table 5.1 below.

Table 5.1

DS1 Category	DS2 Category	DS3 Category	Standardised To
Apparel	Clothing	Clothing & Apparel	Clothing & Apparel
Shoes	-	Clothing & Apparel	Clothing & Apparel
Electronics	Electronics	Electronics	Electronics
Accessories	-	Accessories	Accessories
Home	Home & Kitchen	Home & Furniture	Home & Furniture
Beauty	Personal Care	-	Beauty (no DS3 match)
Sports	Sports & Outdoors	-	Sports (no DS3 match)
Groceries	Groceries	-	Groceries (no DS3 match)

A Formula tool is added on the DS1+DS2 branch after the Select tool to remap DS1 category values to DS3's naming convention as shown in Figure 5.1. The expression updates the existing category field in place rather than creating a new column. Categories with no equivalent in DS3 (Beauty, Sports, Groceries) are left unchanged so these rows will return null values in the DS3 aggregation columns after Join 2, which is the expected outcome of a Left Join.

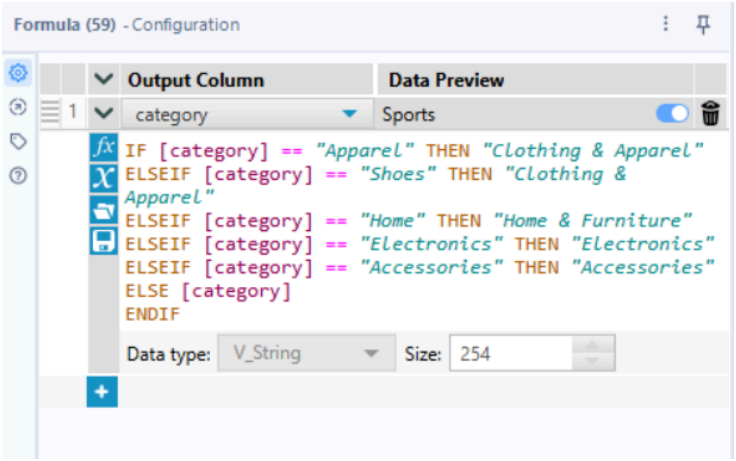

Figure 5.2 *Formula Tool showing category standardisation expression on DS1+DS2*

5.5 DS3 Pre-Processing (Summarise Tool)

DS3 contains one row per customer order, making it an order-level dataset with hundreds of thousands of records. Joining it directly to the DS1+DS2 product-level table would cause row multiplication, as each product record would match multiple order records. To prevent this, DS3 is aggregated to the category and month level using a Summarise tool before the join, the configuration is shown in Figure 5.3 below.

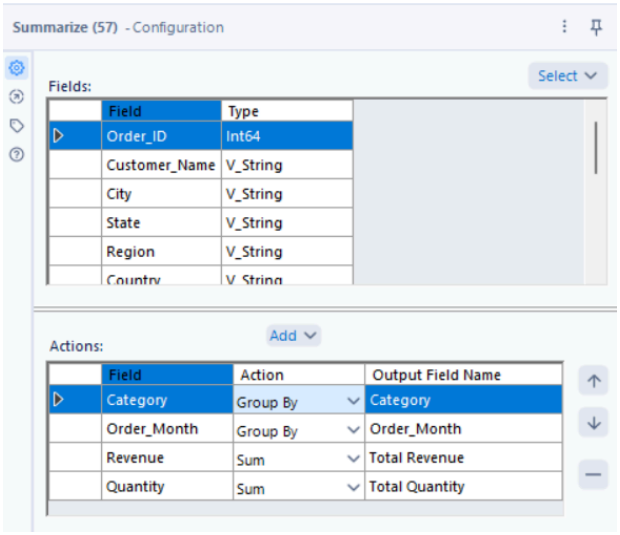

Figure 5.3 *Summarise tool configuration; Group By: Category, Order_Month, SUM: Revenue (→ total_revenue), Quantity (→ total_quantity)*

Prior to summarising, a Formula tool extracts the order month from the Order_Date field. DS3's dates are stored in MM-DD-YY format (e.g. 08-23-23), so the month is extracted using Left([Order_Date], 2) converted to an integer, producing a new field Order_Month. A Select tool is also applied to DS3 to trim leading and trailing spaces from the Revenue and Unit_Price column names, which were found to contain whitespace that would cause silent reference errors in downstream tools.

5.6 Join 2 (DS1 + DS2 merged at Category level, DS3 aggregated)

Join key: category (DS1+DS2 merged) = Category (DS3 aggregated)

Join type: Left Join

The second Join tool merges the DS1+DS2 table with the DS3 summarised table on the category field as displayed in Figure 5.4. The DS1+DS2 merged table is connected to the L (Left) input and the DS3 aggregated table to the R (Right) input. A Left Join is used again to ensure all DS1+DS2 records are retained regardless of whether their category has a matching entry in DS3.

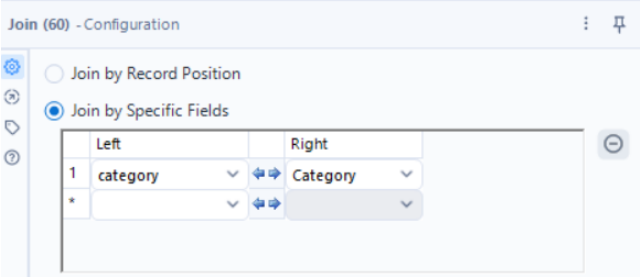

Figure 5.4 *Join 2 configuration panel, join condition: merged.category = DS3_agg.Category*

Following the join, a Union tool combines the J output port (matched rows are categories that exist in both DS1 and DS3) with the L output port (unmatched rows are Beauty, Sports, Groceries categories with no DS3 equivalent) as shown in Figure 5.5. This ensures all 172,791 records flow through to the Browse tool and subsequently to Member 4.

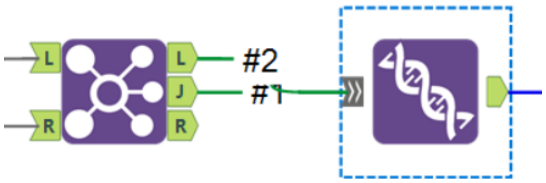

Figure 5.5 *Union tool combining J and L outputs of Join 2*

5.7 Data Flow Diagram

Figure 5.6 below illustrates the complete integration pipeline. DS3 follows a separate pre-processing branch (DateTime → Cleansing → Select → Formula → Summarise) before converging at Join 2. The two join outputs are unified before the Browse tool to preserve all records for downstream transformation.

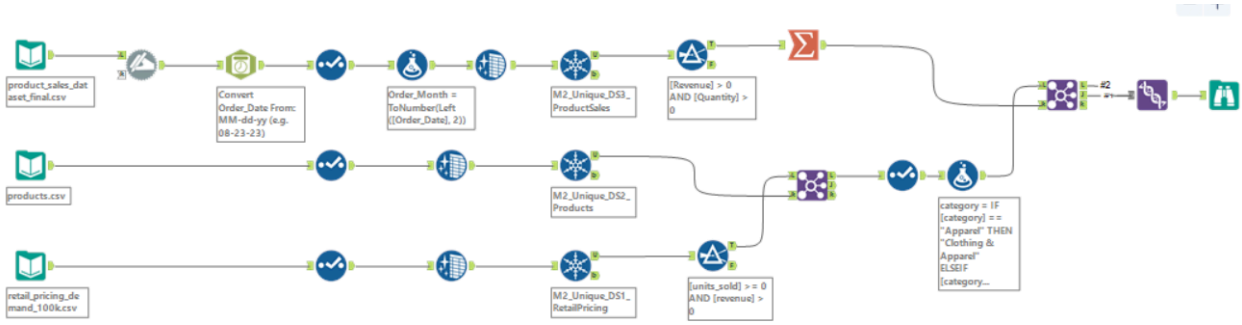

Figure 5.6 *Data flow diagram from DS1 → Join 1 ← DS2 → Select → Formula (category) → Join 2 ← DS3 (Formula + Summarise) → Union → Browse*

5.8 Browse Tool Output

After running the complete integration workflow, the Browse tool confirms the output contains 172,791 records and 21 fields as shown in Figure 5.7. The field count reflects DS1's 17 original columns plus 4 fields from DS3's aggregation (Category, Order_Month, total_revenue, total_quantity). DS2 columns are absent from all rows due to the product_id range mismatch described in Section 5.2.

Browse (63) - SkinAqua_ProjectWorkflow_Phase2.yxmd

172,791 records displayed, 21 fields, 3.8 MB

Table | Profile

21 of 21 Fields | Cell Viewer

Record	date	product_id	category	brand	region	channel	season	base_price	current_price	price_change_pct
1	2026-01-01	P1051	Clothing & Apparel	Puma	AU	app	Winter	150.86	128.23	-15
2	2026-01-01	P1051	Clothing & Apparel	Puma	DE	web	Winter	150.86	120.69	-20
3	2026-01-01	P1051	Clothing & Apparel	Puma	US	app	Winter	150.86	150.86	0
4	2026-01-02	P1051	Clothing & Apparel	Puma	AU	mobile	Winter	150.86	128.23	-15
5	2026-01-02	P1051	Clothing & Apparel	Puma	DE	mobile	Winter	150.86	135.77	-10
6	2026-01-02	P1051	Clothing & Apparel	Puma	US	mobile	Winter	150.86	128.23	-15
7	2026-01-03	P1051	Clothing & Apparel	Puma	AU	app	Winter	150.86	150.86	0
8	2026-01-03	P1051	Clothing & Apparel	Puma	DE	web	Winter	150.86	150.86	0
9	2026-01-03	P1051	Clothing & Apparel	Puma	US	web	Winter	150.86	90.52	-40
10	2026-01-04	P1051	Clothing & Apparel	Puma	AU	mobile	Winter	150.86	150.86	0

discount_pct	promotion_type	units_sold	revenue	inventory_level	stockout_flag	demand_index	Right_Category	Order_Month	Total Revenue	Total Quantity
15	Percentage Discount	10	1282.3	203	False	86.96	Clothing & Apparel	20	27134365.3	115523
20	Flash Sale	19	2293.11	232	False	165.22	Clothing & Apparel	20	27134365.3	115523
0	None	13	1961.18	259	False	113.04	Clothing & Apparel	20	27134365.3	115523
15	Percentage Discount	14	1795.22	334	False	121.74	Clothing & Apparel	20	27134365.3	115523
10	Percentage Discount	13	1765.01	319	False	113.04	Clothing & Apparel	20	27134365.3	115523
15	Percentage Discount	13	1666.99	301	False	113.04	Clothing & Apparel	20	27134365.3	115523
0	None	12	1810.32	278	False	104.35	Clothing & Apparel	20	27134365.3	115523
0	None	13	1961.18	182	False	113.04	Clothing & Apparel	20	27134365.3	115523
40	Clearance	16	1448.32	299	False	139.13	Clothing & Apparel	20	27134365.3	115523

Figure 5.7 *Browse tool results, showing 172,791 records, 21 fields, first few rows with column headers visible*

5.9 Challenges Encountered during Integration Phase

1. Product_id type and range mismatch between DS1 (String: P1001–P1010+) and DS2 (Integer: 1–20). Resolved by recasting DS2's product_id to String in the Select tool to prevent a type error, but the numeric ranges remain incompatible, resulting in zero matched rows in Join 1. All DS1 records exit via the L port with DS2 columns null.
2. Category naming inconsistency across all three datasets, each dataset was authored independently on Kaggle using different category vocabularies. Only Electronics matches exactly across DS1 and DS3. A Formula tool was used to remap DS1 categories to DS3's naming convention before Join 2.
3. DS3 column names contained leading and trailing whitespace (e.g. ' Revenue ', ' Unit_Price '). Identified by inspecting the raw CSV. Resolved using a Select tool to rename affected fields before they were referenced in the Summarise tool.
4. DS3 Order_Date is stored in MM-DD-YY format rather than the standard YYYY-MM-DD used in DS1. The DateTime tool could not parse the two-digit year reliably, so month extraction was performed manually using Left([Order_Date], 2) in a Formula tool.
5. Three DS1 categories, Beauty, Sports, and Groceries have no equivalent in DS3, accounting for approximately 37.5% of DS1 records. These rows return null DS3 columns after Join 2. A Union tool combining the J and L outputs of Join 2 ensures these records are not discarded.

6.0 Data Transformation and Feature Creation

After the datasets were integrated and clean, a Formula tool was used to create seven new columns from the previous fields. These derived columns are what the Phase 3 visuals are built around. Rather than calculating things like monthly groupings or stock categories inside Tableau, everything was already prepared in advance so that the output CSV is already analysis-ready.

6.1 Derived Columns

Table 6.1.1

Column	Source Fields	What It Does
month, year	Date (DS1)	Extracted to support monthly trend grouping across 2023-2024. Splitting them out is cleaner compared to replying on Tableau to parse dates on its own.
discount_amount	Base_price, current price (DS1)	base_price minus current_price gives the actual monetary discount per unit, which is more readable in a dashboard than a percentage alone.
price_sensitivity_flag	Price_change_pct, units_sold (DS1)	Labels observations as “High Sensitivity” once price went up but sales dropped significantly. Makes price-sensitive products easy to spot in the scatter plot.
stock_status	Inventory_level, stockout_flag (DS1)	Converts the raw stockout_flag and inventory numbers into readable labels, which are “Out of Stock”, “Low Stock”, or “In Stock”. Feeds into the treemap visual in Phase 3.

revenue_per_unit	Revenue, Quantity (DS3)	Revenue divided by Quantity shows average value per item sold, independent of volume. A divide-by-zero guard was added for rows where Quantity is 0.
product_tier	Price (DS2)	Group products into Premium, Mid-Range, or Budget based on catalog price. Adds a segmentation dimension for the grouped bar chart comparisons.

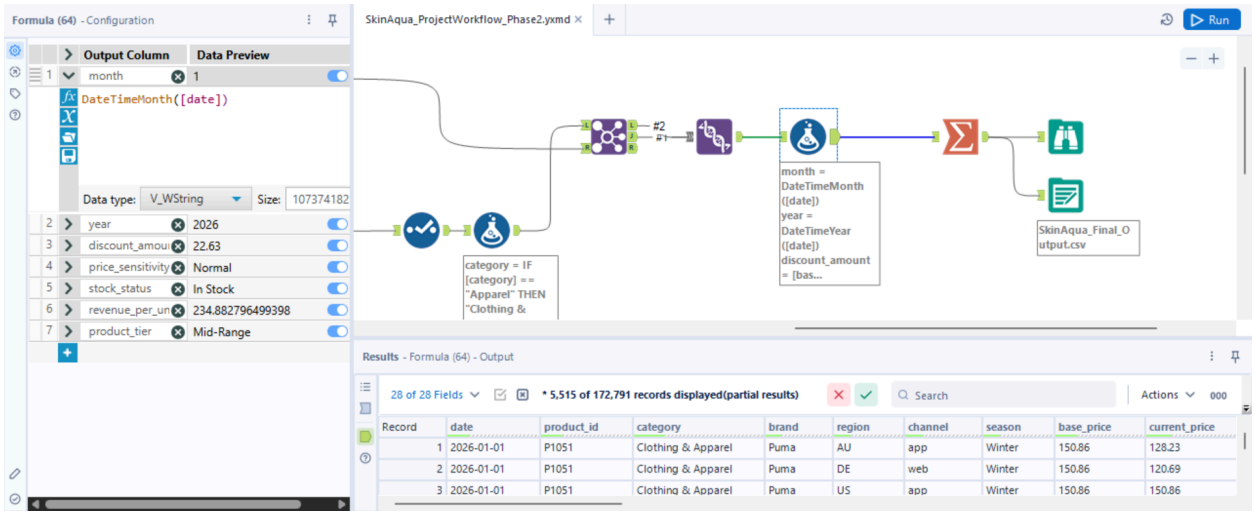

Figure 6.1 *Formula tool config panel showing at least 3 expressions*

6.2 Aggregation and Output

Once all of the derived columns were already ready, a Summarise tool was used to collapse the 172,000-row merged table into a grouped summary. Data was grouped by category, season, region, month, year, and product_tier. The following aggregations were computed within each group:

- Sum: total_revenue, units_sold → total_units_sold
- Average: demand_index → avg_demand_index, discount_amount → avg_discount_amount
- Count: product_id → product_count

The aggregated output was then exported by using the Output Data tool as SkinAqua_Final_Output.csv. After exporting, the file was opened in Excel to do a quick sanity check, to make sure all derived columns exist, and the stock_status and product_tier values worked as expected.

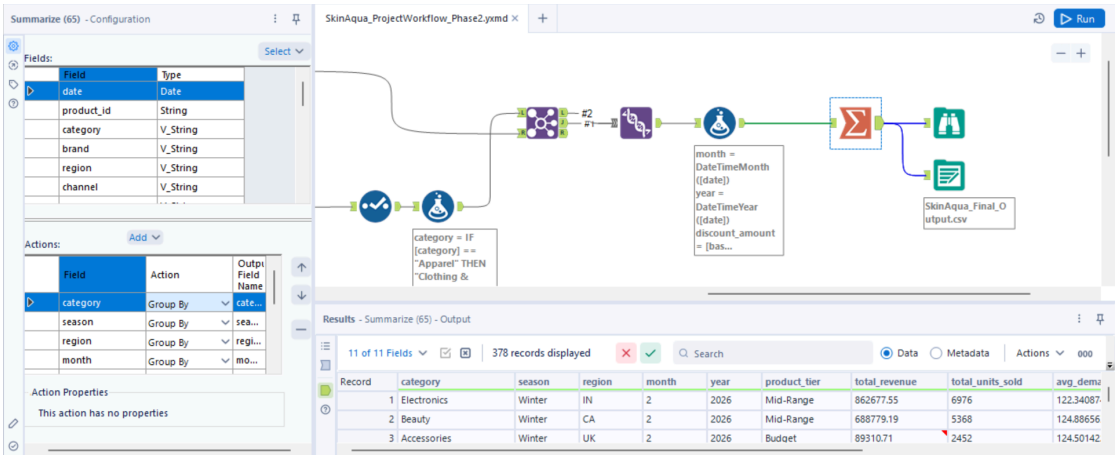

Figure 6.2 *Summarize tool config showing group-by fields and aggregations*

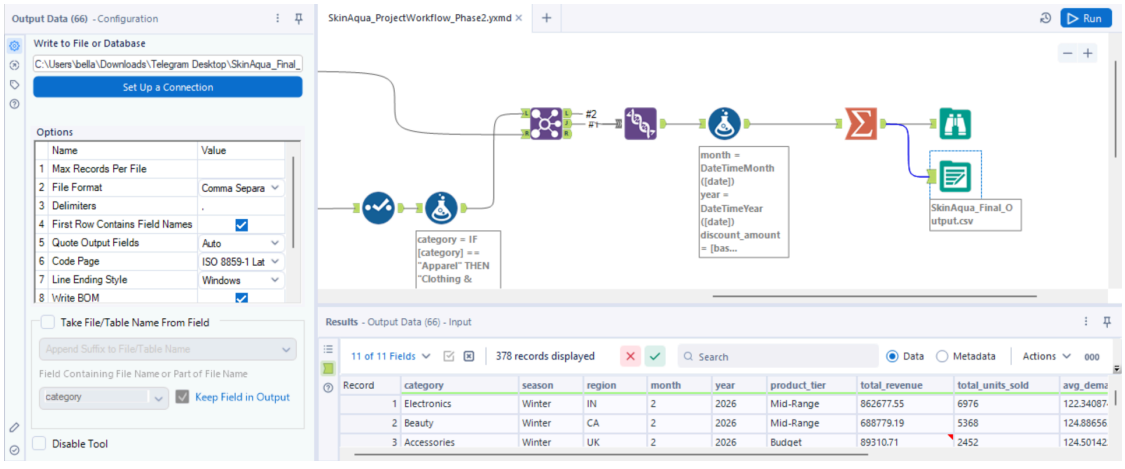

Figure 6.3 *Output Data tool + first few rows of the exported CSV in Excel*

7.0 Challenges Faced

7.1 product_id type mismatch between DS1 and DS2.

DS1 stores product_id as a String, but DS2 has it as an Integer. Alteryx could not match them directly, so the join returned no rows on the first attempt. Fixed by adding a Select tool on the DS1 branch to convert product_id to Integer before the join.

7.2 Category names do not match between DS1 and DS3

DS1 and DS3 come from different Kaggle authors so their category values use different labels. For example, DS1 uses “Personal Care” while DS3 uses “Clothing & Apparel”. A Find Replace tool was used on DS3 to remap its categories to match DS1’s naming before the join.

7.3 stockout_flag stored as string, not Boolean

The stockout_flag field came as the strings “True” and “False” rather than actual Boolean values. The first version of the stock_status formula checked [stockout_flag] = 1 and returned “In Stock” for everything. Updated the condition to [stockout_flag] = “True” and it worked correctly after that.

7.4 Timestamp included in DS3’s date column

Some rows in DS3’s Order_Date column had timestamps appended, which caused the DateTime conversion to fail for those rows. Added a Formula step before the DateTime tool using Left([Order_Date], 10) to strip everything after the date part, which resolved the issue.

8.0 Conclusion

The phase 2 workflow brings all datasets through a complete pipeline which includes cleaning, integration, feature creation, aggregation, and produces a single output file ready for Tableau. The seven derived columns added during transformation are specifically designed to support the dashboard visuals planned for Phase 3, so the data does not need any further processing once it is imported into Tableau. A few data quality issues came up during the build which were mainly type mismatches and inconsistent formatting between datasets from different sources, but all of them were resolved within the workflow. Phase 3 will use this output to build the pricing, demand, and inventory dashboards that the project set out to deliver.